Python Lists: A Reference

> *A list in Python is an ordered, mutable collection of items enclosed in square brackets (`[]`). Lists can store elements of different data types and change size dynamically. Use this handout as a guide for key actions.*

1. Basic Operators & Functions

Tool / Syntax	Instruction & Core Mechanics	Step-by-Step Code & Output

`+` (Concatenation)	Merges two independent lists into a completely new list without changing the original variables.	`>>> list1 = [1, 2]`
		`>>> list2 = [3, 4]`
		`>>> combined = list1 + list2`
		`>>> combined`
		`[1, 2, 3, 4]`

`*` (Duplication)	Repeats the sequence of items inside a list `N` times. Useful for establishing blank variables or default sequences.	`>>> baseline = [0] * 4`
		`>>> baseline`
		`[0, 0, 0, 0]`
		`>>> ["Yes", "No"] * 2`
		`['Yes', 'No', 'Yes', 'No']`

`len()`	Evaluates the list and counts the total items inside it. Always returns a whole number (integer).	`>>> shopping = ["milk", "eggs"]`
		`>>> total_items = len(shopping)`
		`>>> total_items`
		`2`

`in`	Performs a linear scan to check if a single item matches any element in the list. Returns `True` or `False`.	`>>> tools = ["hammer", "saw"]`
		`>>> "saw" in tools`
		`True`
		`>>> "drill" in tools`
		`False`

2. Adding & Inserting Items

Method	Instruction & Core Mechanics	Step-by-Step Code & Output

`.append(item)`	Appends a single item to the very end of the target list. Directly modifies the original variable in memory.	`>>> queue = ["Alice", "Bob"]`
		`>>> queue.append("Charlie") # Modifies list`
		`>>> queue`
		`['Alice', 'Bob', 'Charlie']`

`.insert(i, item)`		`>>> track = ["Lap 1", "Lap 3"]` `>>> track.insert(1, "Lap 2") # Put at index 1`

Method	Instruction & Core Mechanics	Step-by-Step Code & Output
	Forces an item into index position `i`. Shifts all subsequent items downstream to the right.	`>>> track` `['Lap 1', 'Lap 2', 'Lap 3']`

3. Removing Items (Three Strategies)

Method	Instruction & Core Mechanics	Step-by-Step Code & Output

`.remove(val)`	Removes the first occurrence of a specific value. Throws a `ValueError` error if the value doesn't exist.	`>>> stock = ["apple", "pear", "apple"]`
		`>>> stock.remove("apple") # Removes 1st match`
		`>>> stock`
		`['pear', 'apple']`

`.pop(index)`	Removes and isolates an item at a specific index. If no index is provided, it safely extracts the final element.	`>>> items = ["low", "mid", "high"]`
		`>>> priority = items.pop() # Pulls from end`
		`>>> items`
		`['low', 'mid']`
		`>>> priority`
		`'high'`

`.clear()`	Wipes out every item in the list, returning it back to an empty placeholder state (`[]`).	`>>> workspace = [102, 504, 301]`
		`>>> workspace.clear()`
		`>>> workspace`
		`[]`

4. Reordering & Inspecting

Method	Instruction & Core Mechanics	Step-by-Step Code & Output

`.index(val)`	Searches left-to-right and returns the exact integer position index where the target value first appears.	`>>> roster = ["Sam", "Max", "Mia"]`
		`>>> roster.index("Max")`
		`1`

`.count(val)`	Scans the entire collection and counts the number of times a target value is found.	`>>> votes = ["Yes", "No", "Yes"]`
		`>>> votes.count("Yes")`
		`2`

`.sort()`	Reorders list contents permanently (alphabetical or numerical ascending). Must contain comparable types.	`>>> points = [15, 5, 10]`
		`>>> points.sort()`
		`>>> points`
		`[5, 10, 15]`

`.reverse()`		`>>> setup = [1, "two", 3]` `>>> setup.reverse()`

Method	Instruction & Core Mechanics	Step-by-Step Code & Output
	Flips the array layout end-to-end. Does not sort data backwards; it simply reads it right-to-left.	`>>> setup` `[3, 'two', 1]`